```
; Write X86/64 ALP to perform non-overlapped block transfer without string specific instructions.

; Block containing data can be defined in the data segment.


; ---------- Macros ----------

; Macro for write syscall (write to stdout)
%macro write 2
        mov rax, 1          ; syscall number for write
        mov rdi, 1          ; file descriptor (stdout)
        mov rsi, %1         ; buffer address
        mov rdx, %2         ; length
        syscall
%endmacro

; Macro for read syscall (read from stdin)
%macro read 2
        mov rax, 0          ; syscall number for read
        mov rdi, 0          ; file descriptor (stdin)
        mov rsi, %1         ; buffer address
        mov rdx, %2         ; length
        syscall
%endmacro

; ---------- Data Section ----------

section .data
        ; Display introduction message
        dispMsg db "Write X86/64 ALP to perform overlapped block transfer with string specific instructions Block containing data can be defined in the data segment.",10, "Name:- Sumaanyu",10,"Roll:- 7256",10,"Date of Performance:- 24/03/2025",10
        dispMsgLen equ $-dispMsg

        ; Menu string
        msg db "----------MENU----------", 10
          db "      1. Without string instruction ", 10
          db "      2. With string instruction ", 10
          db "      3. Exit ", 10
          db "Enter your choice : "
        msglen equ $-msg

        msg1 db "Number : "
        len equ $-msg1

        endl db 10          ; newline character

        src dq 10h, 20h, 30h, 40h, 50h ; Source block of 5 quadwords
        dest dq 0h, 0h, 0h, 0h, 0h    ; Destination block initialized to 0

        tab db 20h          ; tab (space)

        source db "    Source      Destination", 10
        sourcelen equ $-source

        b db "Before : ", 10      ; label before transfer
        a db "After  : ", 10      ; label after transfer
        l equ $-a

; ---------- BSS Section ----------

section .bss
        temp resb 16      ; buffer for hex string
        choice resb 2     ; for reading menu choice

; ---------- Text Section ----------

section .text
        global _start

_start :
        write dispMsg, dispMsgLen     ; print introduction
```

```
main_menu :
        write msg, msglen          ; display menu
        read choice, 2             ; read user input

        cmp byte[choice], "1"      ; compare with '1'
        je case1                   ; if equal, jump to case1

        cmp byte[choice], "2"
        je case2

        cmp byte[choice], "3"
        je case3                   ; exit

case1 :
        ; Case 1: Without string instructions
        write b, l                 ; display "Before:"
        mov rcx, 5
        mov rsi, dest
        mov rax, 0

init1 :
        mov [rsi], rax             ; set dest[i] = 0
        add rsi, 8
        loop init1

        call displayBlock          ; show initial blocks
        write endl, 1
        write a, l                 ; display "After:"

        ; Manual block transfer loop
        mov rcx, 5
        mov rsi, src
        mov rdi, dest

next :
        mov rax, [rsi]             ; copy src[i] to dest[i]
        mov [rdi], rax
        add rsi, 8
        add rdi, 8
        loop next

        call displayBlock          ; show updated blocks
        write endl, 1
        jmp main_menu

case2 :
        ; Case 2: With string instruction (movsq)
        write b, l
        mov rcx, 5
        mov rsi, dest
        mov rax, 0

init2 :
        mov [rsi], rax
        add rsi, 8
        loop init2

        call displayBlock          ; show initial state

        write endl, 1
        write a, l

        mov rcx, 5
        mov rsi, src
        mov rdi, dest
        cld                        ; clear direction flag (forward
direction)

next2 :
        movsq                      ; copy qword from [rsi] to [rdi]
        loop next2
```

```
        call displayBlock       ; show updated blocks
        write endl, 1
        jmp main_menu

case3 :
        ; Exit program
        mov rax, 60
        mov rdi, 0
        syscall

; -------------------- displayBlock Procedure --------------------

displayBlock :
        mov rcx, 5
        mov rsi, src

nextD :
        push rcx
        push rsi

        mov rbx, [rsi]          ; load source element
        call display            ; display it
        write tab, 1

        pop rax
        mov rbx, rax
        push rax
        call display            ; display same value again
(redundant but preserved)
        write endl, 1

        pop rsi
        pop rcx
        add rsi, 8
        loop nextD

        write endl, 1

        mov rcx, 5
        mov rdi, dest

nextA :
        push rcx
        push rdi

        mov rbx, [rdi]          ; load destination element
        call display
        write tab, 1

        pop rax
        mov rbx, rax
        push rax
        call display
        write endl, 1

        pop rdi
        pop rcx
        add rdi, 8
        loop nextA
        ret

; -------------------- display Procedure --------------------

display :
        mov rsi, temp           ; point to temp buffer
        mov rcx, 16             ; each qword = 16 hex digits

next1 :
        rol rbx, 4              ; rotate to get next nibble
        mov al, bl
        and al, 0Fh             ; isolate nibble
        cmp al, 9
        jbe add30h
```

```
                add al, 7h              ; for 'A'-'F'

add30h :
                add al, 30h             ; convert to ASCII
                mov [rsi], al
                inc rsi
                loop next1

                write temp, 16          ; print the full hex
                ret
```